

Subqueries en WHERE

Hasta ahora, cada vez que filtramos datos con WHERE, usamos valores que ya conocíamos de antemano. Algo como `WHERE pais = 'Argentina'` o `WHERE total > 500`. Son filtros estáticos: tú sabes el valor y lo escribes directamente.

Pero hay preguntas de negocio donde el valor del filtro no lo sabes de antemano, porque depende de los propios datos. Por ejemplo: *"¿Qué clientes hicieron pedidos por más que el promedio general de ventas?"*. No puedes escribir `WHERE total > 1837` porque ese promedio cambia cada vez que se registra una nueva venta. Lo que necesitas es que SQL calcule ese promedio y lo use como filtro dentro de la misma consulta. Eso es exactamente lo que hace una subquery.

Una query dentro de otra query

El concepto es más simple de lo que suena. Una subquery es literalmente una consulta metida dentro de otra consulta. La de afuera se llama **query principal** y la de adentro se llama **subquery**. Lo importante es entender el orden en que se ejecutan: primero corre la subquery, produce un resultado, y después la query principal usa ese resultado como si fuera un valor más.

Piénsalo como un proceso de dos pasos que SQL hace automáticamente por ti. Paso uno: "calcula el promedio de todas las ventas completadas". Paso dos: "ahora dame todos los clientes cuyas compras sean mayores a ese número". La subquery es el paso uno, y la query principal es el paso dos. La gracia es que ambos pasos viven dentro de una sola consulta.

Ahora, no todas las subqueries funcionan igual. Hay tres tipos y cada uno responde a una necesidad distinta.

Subquery escalar:

La más directa es la subquery escalar. "Escalar" simplemente significa que devuelve **un único valor**: un número, una fecha, un texto. Uno solo.

El ejemplo clásico es el que mencionamos al inicio: encontrar clientes que compraron por encima del promedio. La subquery calcula el promedio general de los

pedidos completados — digamos que da 1,837 — y la query principal usa ese número como umbral en el WHERE para filtrar.

¿Podrías calcular el promedio por separado, anotarlo y escribirlo a mano en el WHERE? Técnicamente sí. Pero la diferencia es importante: si mañana entran nuevos pedidos y el promedio cambia, la subquery siempre va a usar el valor actualizado. El número escrito a mano queda viejo. La subquery mantiene tu consulta viva y dinámica.

Subquery de lista:

Ahora supongamos que la pregunta es: “¿Qué productos se vendieron al menos una vez en Argentina?”. Acá el filtro no es un número contra el cual comparar, es una **lista** de productos — todos los que se vendieron en Argentina — y necesitas saber cuáles de tu catálogo aparecen en esa lista.

Para esto existe la subquery de lista, que se usa con el operador **IN**. La subquery te devuelve una columna con múltiples valores (en este caso, los IDs de productos vendidos en Argentina), y la query principal dice: “de todos mis productos, muéstrame solo los que estén IN esta lista”.

En la clase vimos que TiendaLatam tiene 34 productos en su catálogo, la subquery encuentra 28 que se vendieron en Argentina, y la consulta completa te devuelve los 24 que cumplen todas las condiciones. Cada pieza hace su trabajo por separado, pero juntas responden la pregunta de negocio.

Y así como **IN** te dice “lo que está en la lista”, **NOT IN** te dice “lo que no está”. Por ejemplo: “¿Qué productos nunca se vendieron?”. Misma lógica, pero invertida.

Sin embargo, **NOT IN** tiene una trampa que necesitas conocer. Si la subquery devuelve aunque sea un solo valor **NULL** en la lista, **NOT IN** deja de funcionar como esperas: te devuelve cero filas, incluso cuando claramente debería haber resultados. Es un comportamiento de SQL que sorprende hasta a gente con experiencia. La solución es agregar un **WHERE ProductoID IS NOT NULL** dentro de la subquery para limpiar los **NULLs** antes de que lleguen al **NOT IN**. O mejor aún, usar **EXISTS**, que es lo que viene ahora.

EXIST:

EXISTS es un tipo especial de subquery que se llama **subquery correlacionada**. La diferencia con las anteriores es que esta no se ejecuta una sola vez y listo — se ejecuta **una vez por cada fila** de la query principal, porque necesita revisar cada registro individualmente.

La pregunta que responde EXISTS es simple: *"¿Existe al menos una fila que cumpla esta condición?"*. No te devuelve valores ni listas, solo responde sí o no. Si existe al menos una fila, la condición es verdadera y esa fila de la query principal se incluye en el resultado.

Por ejemplo: *"¿Qué clientes hicieron al menos un pedido por más de \$500?"*. La subquery revisa, para cada cliente, si existe algún pedido suyo mayor a \$500. Si lo encuentra, ese cliente entra en el resultado. Si no, queda fuera.

Un detalle que vas a ver en el código es que dentro del EXISTS la subquery dice **SELECT 1**. Eso no significa que estés buscando el número uno. Es una convención. Como EXISTS solo necesita saber si hay filas o no, da igual qué columna pongas en el SELECT — lo único que importa es si la consulta devuelve algo. Poner **SELECT 1** es la forma estándar de decir "no me importa el qué, solo me importa que exista".

Y acá viene algo útil: **NOT EXISTS** es la alternativa segura al **NOT IN** que mencionamos antes. Hace lo mismo — encontrar lo que no tiene par — pero no tiene el problema de los NULLs. Si en la clase anterior aprendiste el patrón de LEFT JOIN + WHERE IS NULL para encontrar registros huérfanos, NOT EXISTS es otra forma de llegar al mismo resultado, y en algunos casos es más legible.

¿Y cuándo uso cada tipo?

La lógica es bastante intuitiva una vez que la ves. Si tu filtro depende de **un solo valor calculado** (un promedio, un máximo, una fecha), subquery escalar. Si depende de **una lista de valores** (IDs de productos, códigos de país), subquery con IN. Y si solo necesitas saber **si algo existe o no** para cada fila, EXISTS.

Las subqueries en WHERE abren un nivel de consultas que los filtros estáticos simplemente no pueden alcanzar. Y esto es solo el principio: en las próximas clases vamos a ver qué pasa cuando las subqueries se mueven al FROM y al SELECT, que son dos lugares completamente distintos con propósitos completamente distintos.

Desafío de cierre

Escribe una consulta que encuentre los clientes de TiendaLatam que realizaron su primera compra en el mismo mes que la venta más grande de la historia de TiendaLatam. Usa subqueries en WHERE para resolverlo. Después, reescribe la misma lógica usando EXISTS. Pregúntate: ¿hay diferencia en el resultado? ¿Y en la legibilidad? Comparte ambas versiones y tu análisis en los comentarios de la clase.